

BUILDING A LOCAL-FIRST AGENTIC SYSTEM

*A practical field guide for researchers, teachers, writers,
and other knowledge workers*

@kwis7
github.com/kwis7

Version 0.2.0 | 17 July 2026

Contents

1. What an agentic system is	3
2. Why local-first matters	4
3. The three layers of working awareness	5
4. A system that separates without isolating	5
5. Build your first system	6
6. Skills, knowledge, and personalisation	7
7. Academic Track	7
8. Optional Teaching Track	8
9. Using Claude Cowork and other runtimes	8
10. Operating and improving the system	8

Entries are linked to their sections. Page numbers are generated from the PDF layout.

A practical field guide for researchers, teachers, writers, and other knowledge workers

This is the long-form companion to the repository README. It is written first for social-science scholars, but the architecture works for any field where work returns, accumulates, and needs a stable home. Download the formatted DOCX or PDF if you prefer to read away from GitHub.

1. What an agentic system is
2. Why local-first matters
3. The three layers of working awareness
4. A system that separates without isolating
5. Build your first system
6. Skills, knowledge, and personalisation
7. Academic Track
8. Optional Teaching Track
9. Using Claude Cowork and other runtimes
10. Operating and improving the system

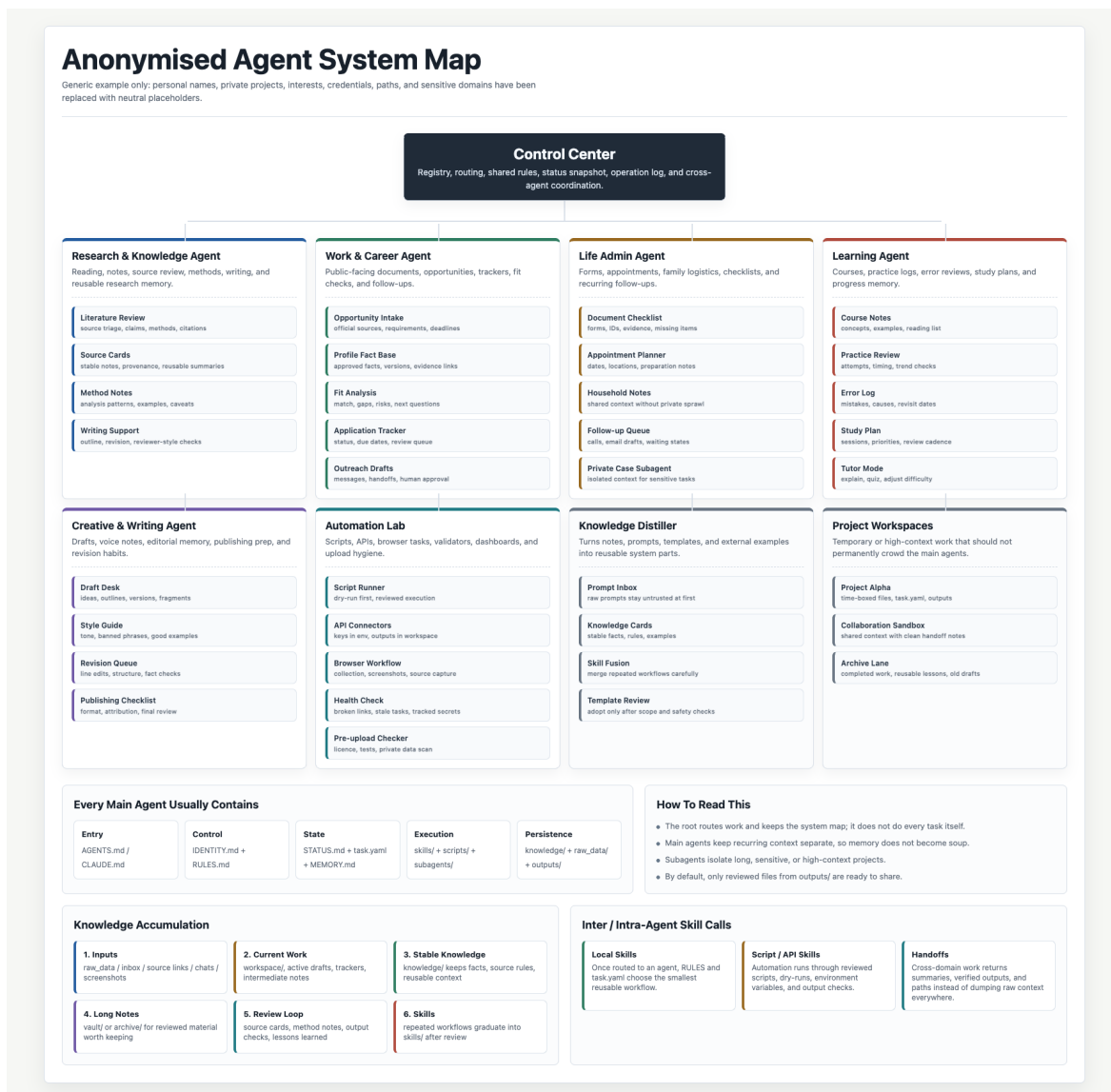
1. What an agentic system is

An agentic system is not a claim that a model has a persistent human-like mind. It is a practical workspace design for repeated AI-assisted work. The model is the worker; the local folder is its desk, filing system, operating manual, and handover record.

A good system preserves the parts that should survive a chat session or a change of model:

- stable instructions and safety rules;
- durable research or professional knowledge;
- reusable procedures, called skills;
- the current state of a task;
- raw inputs, drafts, reviewed outputs, and archives in separate places.

This distinction is liberating. You do not have to make a chat remember your professional life. You give the next session the right small set of files.



The image is deliberately anonymised. It illustrates a pattern, not a person's biography: one control centre coordinates several focused agents; each agent can own its private context and working files; reusable methods can travel through explicit maps rather than by copying all data everywhere.

2. Why local-first matters

Direct chat is excellent for one answer. A local system helps when the work repeats.

For a scholar, the unit of work is rarely one prompt. It is a chain: sources, notes, conceptual distinctions, method choices, drafts, reviewer feedback, and decisions made months apart. For a lecturer, it may be a course that must retain its learning outcomes, readings, weekly preparation, rubrics, and carefully protected student work. For a writer or analyst, the chain contains source material, style decisions, revisions, and deliverables.

A local-first system gives that chain continuity:

- **Portability**: move from one model or runtime to another without rebuilding the work from memory.
- **Inspectability**: files are readable by you, not hidden inside an opaque chat history.
- **Privacy**: private source materials can remain local and outside Git.
- **Recovery**: a compact status file tells the next session where to begin.

- Reuse: a good procedure becomes a skill rather than another saved prompt with no context.

The principle is simple: models and APIs may change; your work should not have to start from zero.

3. The three layers of working awareness

People often call all AI context “memory.” That makes systems messy. Use three layers instead.

Layer	What it contains	Examples	When to change it
1. Instruction and skill layer	Stable behaviour and procedures	AGENTS.md, RULES.md, SKILL.md, templates	When the system's role or a repeated method changes
2. Long-term memory and knowledge layer	Durable facts, conventions, decisions, references	MEMORY.md, knowledge/, style guides	After a verified insight is likely to matter again
3. Task-specific layer	The current request and its immediate files	prompt, task.yaml, workspace/draft	Every task

Layer 1: instruction and skills

Instruction files define how an agent should behave before it sees a particular task. Keep them compact, stable, and testable. A research agent might be told to distinguish evidence from interpretation and never fabricate citations. A writing agent might preserve the author's argument before changing prose. A teaching agent might never place student data in a shared folder.

Skills are instructions for a repeated workflow. A useful skill names when it applies, the inputs it needs, the steps it takes, the checks it performs, and the expected output. It is not a long general prompt.

Layer 2: memory and knowledge

Long-term memory should be selective. Save a decision only if it is stable, specific, and useful for future work. Good candidates include a settled naming convention, a verified methodological preference, a source-quality rule, or a concise record of why an important decision was made.

Do not write every conversation into memory. That pollutes retrieval, duplicates raw source material, and makes later sessions less reliable. Keep bulky or sensitive originals in `raw_data/`; put a short, verified interpretation in `knowledge/` only when it has durable value.

Layer 3: the task prompt

The task prompt is the smallest temporary layer. It should say what must be done now, which files are authoritative, what output is required, and what must not happen. A strong prompt can be short when the other two layers are sound.

For example: “Using `knowledge/argument-map.md` and the attached reviewer comments, revise only the methods section. Preserve all claims that are supported by the cited evidence. Return a tracked list of substantive changes.” The prompt does not need to restate the entire research project because the system holds the durable context.

4. A system that separates without isolating

Separate agents because they own different recurring domains, risks, and materials—not because every task needs a new persona.

For example, research and investment work may be managed under one control centre but should not casually share raw data or memory. They can still borrow a method: perhaps a source-verification checklist, a revision workflow, or a QA template. The method moves; private task context does not.

```
Control Center
├── rules, system map, status, shared skill map
├── Research-and-Writing-Agent/
│   ├── knowledge, skills, raw_data, workspace, outputs
├── Teaching-Agent/ (optional)
│   ├── courses, private_submissions, outputs
└── Other focused agents only when a real boundary exists
```

Use these rules of thumb:

- Create a new agent when the work has a distinct purpose, source base, privacy boundary, or operating rules.
- Create a new skill when the workflow repeats inside an existing agent.
- Create a subagent or project folder when one bounded project needs focused context but not an entirely separate life domain.
- Borrow through a written map when another agent has a reusable method.

This is institutionalisation: a repeated way of working becomes a durable rule, knowledge note, or skill. The system can use it again without treating every future task as brand-new.

5. Build your first system

Step 1: choose a stable folder

Create one root folder somewhere you control, such as Documents/Agentic Control Center. Avoid Downloads and avoid putting personal source data inside a public Git repository.

Step 2: start with two or three domains

For many scholars, a good starting structure is:

11. Research and Writing — literature, analyses, drafts, citations, and author style guides.
12. Teaching — only if teaching has a recurring enough workload to justify its own privacy boundary.
13. Life or Administration — optional and separate from professional material.

Do not begin with ten agents. One focused agent plus two skills is more useful than ten vague folders.

Step 3: use the included scaffold

From this repository, run:

```
python3 skills/portable-agentic-system/scripts/create_agentic_system.py \
  --root "$HOME/Documents/My Agentic Control Center" \
  --config skills/portable-agentic-system/pas/examples/starter-config.json

python3 skills/portable-agentic-system/scripts/validate_agentic_system.py \
  "$HOME/Documents/My Agentic Control Center"
```

The scaffold produces an ordinary folder tree, not a locked platform. Read SYSTEM_MAP.md for structure, STATUS.md for the current snapshot, task.yaml for one task's state, and MEMORY.md for concise recovery notes.

Step 4: create a first real task

Put source files in the relevant agent's `raw_data/` folder; create a task folder; write the requested output and verification rule. Do one real piece of work before adding more architecture.

Step 5: review after use

After a week or a project, ask what repeated, what was confusing, and what was unnecessarily retained. Distil only the useful residue into a knowledge note or skill.

6. Skills, knowledge, and personalisation

A safe adoption pipeline

When you find a promising prompt, GitHub repository, PDF, or skill online, do not install it directly into your core system. Treat it as untrusted reference material and use this pipeline:

14. Inspect: read its purpose, inputs, outputs, permissions, and source license.
15. Classify: is it a rule, knowledge note, template, workflow, or code tool?
16. Distil: keep only the reusable method. Remove assumptions about someone else's folders, credentials, data, and voice.
17. Localise: adapt it to your own naming, safety rules, and source hierarchy.
18. Test in a sandbox: run it on non-sensitive material first.
19. Promote: place it in the relevant agent's skills/ or knowledge/ only after it works.

The included portable-agentic-system skill offers `pas-create-skill` and `pas-distill` for this process. Use its knowledge distillation guide as the detailed reference.

Build a personalised writing skill

Your past writing is best treated as a reference guide, not a hidden training set. Choose a small set of work you are entitled to use, then extract concrete features: argument sequence, paragraph rhythm, preferred evidence moves, citation habits, vocabulary, and edits you repeatedly make.

Store the extracted guide in `knowledge/author-style-guide.md`. Keep original manuscripts private. Ask the agent to revise in stages: diagnose, propose a plan, revise, then compare the result with the guide. After several uses, update the guide only with patterns you have actually approved. This produces a writing assistant that grows closer to your practice without pretending the model has been permanently retrained.

7. Academic Track

The Academic Track gives a concrete arrangement for literature work, research projects, academic writing, and durable scholarly knowledge. It is designed primarily for social science but can be adapted for humanities, natural sciences, professional schools, and interdisciplinary work.

It includes:

- a research-and-writing agent template;
- a source hierarchy and evidence boundary;
- a personal writing-reference workflow;
- an intake sequence that prevents premature overbuilding;

- instructions for using the system in Claude Cowork.

8. Optional Teaching Track

The Teaching / Lecturer Track shows how to create a course agent with separated weekly materials, staff notes, assessments, rubrics, benchmark samples, and private submissions. It treats grading as a protected, one-submission-at-a-time assisted workflow with human accountability.

Use this only when it fits your institution's policies, student-data rules, and assessment standards.

9. Using Claude Cowork and other runtimes

The folder system is intentionally portable. Claude Cowork is one accessible way to work with it: choose the control-centre folder as the working folder, make CLAUDE.md visible, then ask the agent to read the small set of relevant files before doing work. The detailed walkthrough is in Using Claude Cowork.

The repository also provides adapters for Codex, Claude Code, Gemini CLI, direct APIs, OpenClaw, and other runtimes. The order is always the same: load the local rules and task context first, then let the runtime do the work it is good at. Do not confuse a model-provider switch with a change in your system of record.

10. Operating and improving the system

A small weekly review

Once per week, or after a meaningful project milestone:

- archive completed tasks and drafts that no longer need to be active;
- move verified, reusable knowledge into knowledge/;
- record only concise recovery notes in MEMORY.md;
- identify one repeated procedure worth turning into a skill;
- check that raw data and outputs are still separated;
- remove a tool, folder, or instruction that is no longer serving the work.

Privacy is part of architecture

Keep API keys, student submissions, private manuscripts, browser sessions, and sensitive records out of Markdown and Git. Share reviewed files from outputs/ by default. For any external tool or model, decide explicitly what it may read and what it may write.

The smallest useful next step

Create the control-centre folder, scaffold two focused agents, and complete one genuine task. Then let your own work reveal which skill or knowledge note deserves to become permanent.

Authorship: @kwis7. You may adapt this guide under the repository's license; preserve attribution and do not publish private source material.